

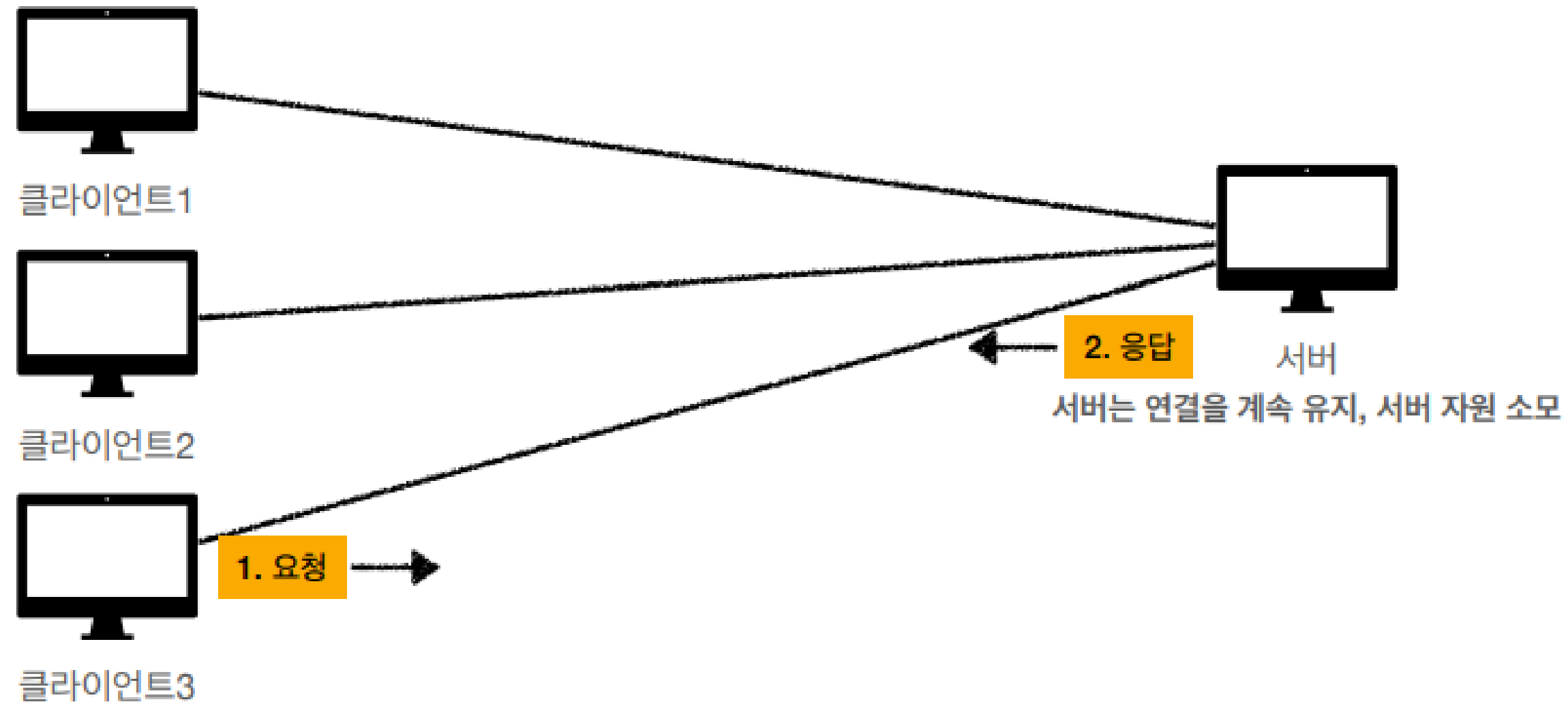
Leets 백엔드 6주차 세션

인증 & 인가

2025.10.30

인증 & 인가

HTTP의 특성



HTTP의 중요한 특성 - Stateless

매 요청을 기억하지 않는다 → 서버의 부하 방지

무엇이 문제냐,,

👤 클라이언트: 안녕하세요, 로그인할게요. 제 이름은 지영입니다.

💻 서버: 반갑습니다. 인증되었습니다.

👤 클라이언트: 좋아요, 내 정보 보여주세요.

💻 서버: 누구시죠?

→ 서버가 클라이언트의 상태를 기억하지 않는다ㅠㅠ

나: 아니 저번에 내가 말했잖아
기억 안 나? 내 말에 대답까지 했었잖아



인증 ? 인가

1. 우리 서비스에 **가입된 유저**인가?? 이 신뢰할 수 있는 사람인가?? → **인증**
2. 요청하는 사람(혹은 컴퓨터?) 이 **특정 자원에 접근** 가능한가?? → **인가**

ex) 3번 유저가 작성한 게시글은 3번 유저만 수정 혹은 삭제할 수 있어야겠죠

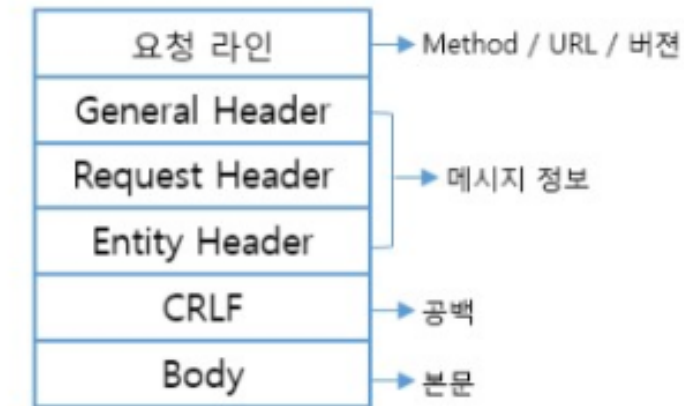
→ 위 두 조건을 만족해야 서버는 적절한 response를 내려줄 수 있음



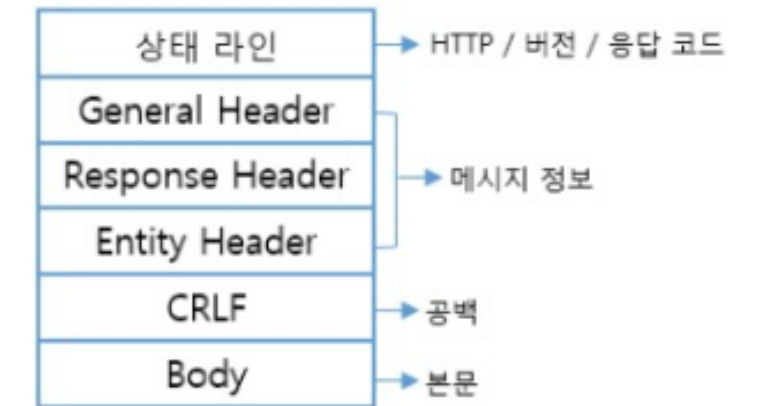
- userId값을 RequestParam이나 헤더로 받음,,, 활짝 노출!! 나 털어가세요..
- 아이디, 비번 → 어떻게 줄건데???!!!

Request Header

HTTP Request Message

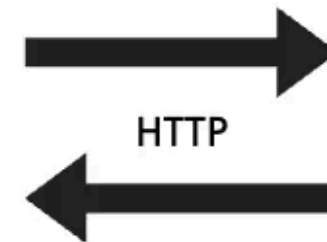
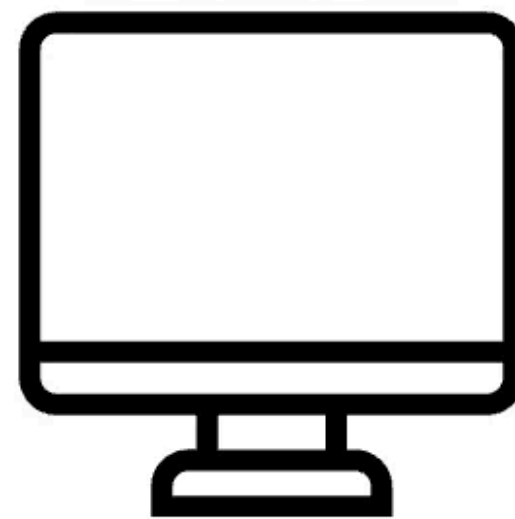


HTTP Response Message

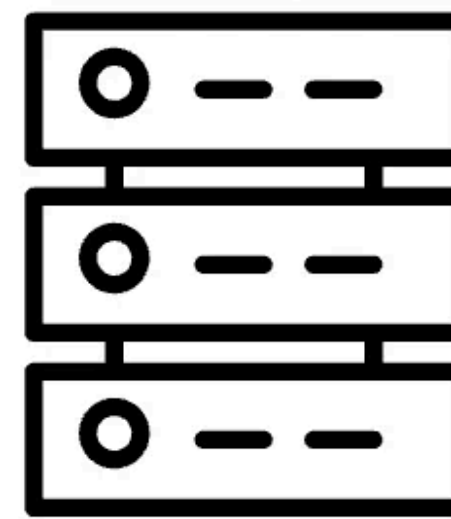


http://user:1234@localhost:8080/login

클라이언트



서버



우리는 앞으로 인증 정보를 헤더에 담아서 보내줄거임→ 이걸 당연한 표준!

우리 서비스에 가입된 유저인가?? 이 신뢰할 수 있는 사람인가? 🤔?

→ 인증

누가 요청을 보냈는가?
그 요청을 믿을 수 있는가?
중간에 누가 그 요청을 엿보고 있지는 않은가?

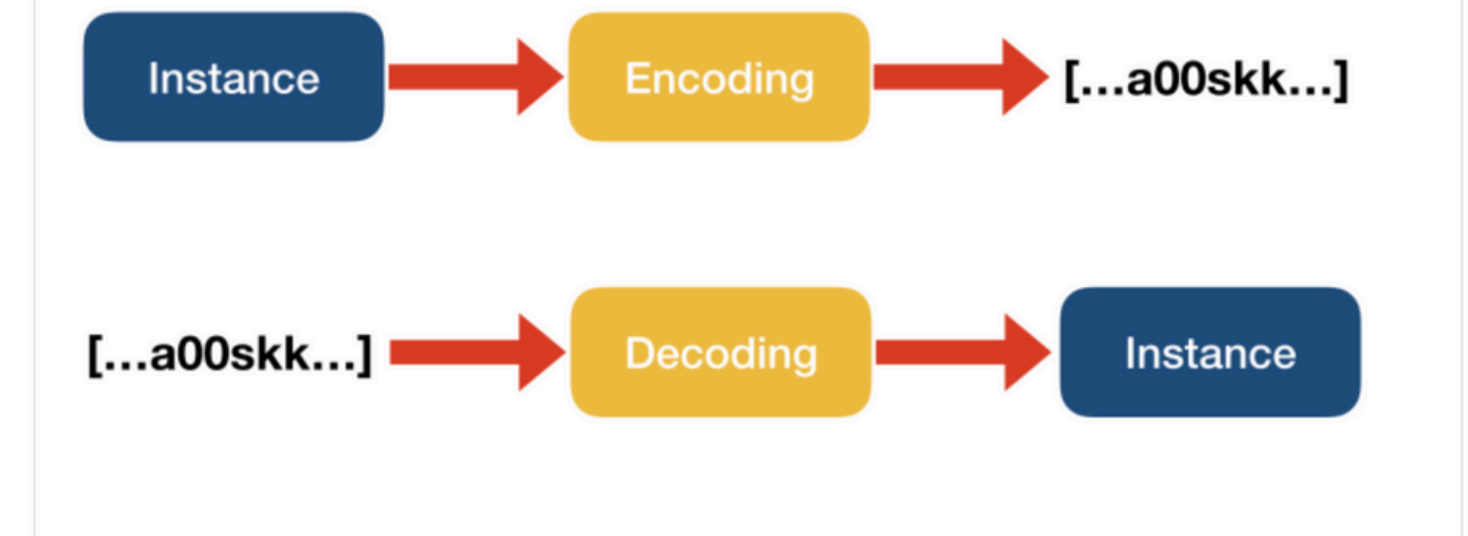
Basic Authentication

그냥 인증 방법 중 하나

Authorization: Basic <인코딩된 문자열>

인코딩 : 데이터를 한 형태(표현 방식)에서 다른 형태로 바꾸는 것

디코딩 : 인코딩된 데이터를 다시 원래의 형태로 되돌리는 것.



1. leetsUserID:leets1234 라는 문자열을 만든 다음

2. 이 문자열을 **base64** 로 인코딩하여 다음과 같은 값을 얻어냅니다

Base64는 안전한 문자 집합(A-Z, a-z, 0-9, +, /)만
사용하므로 전송 중 손상 방지

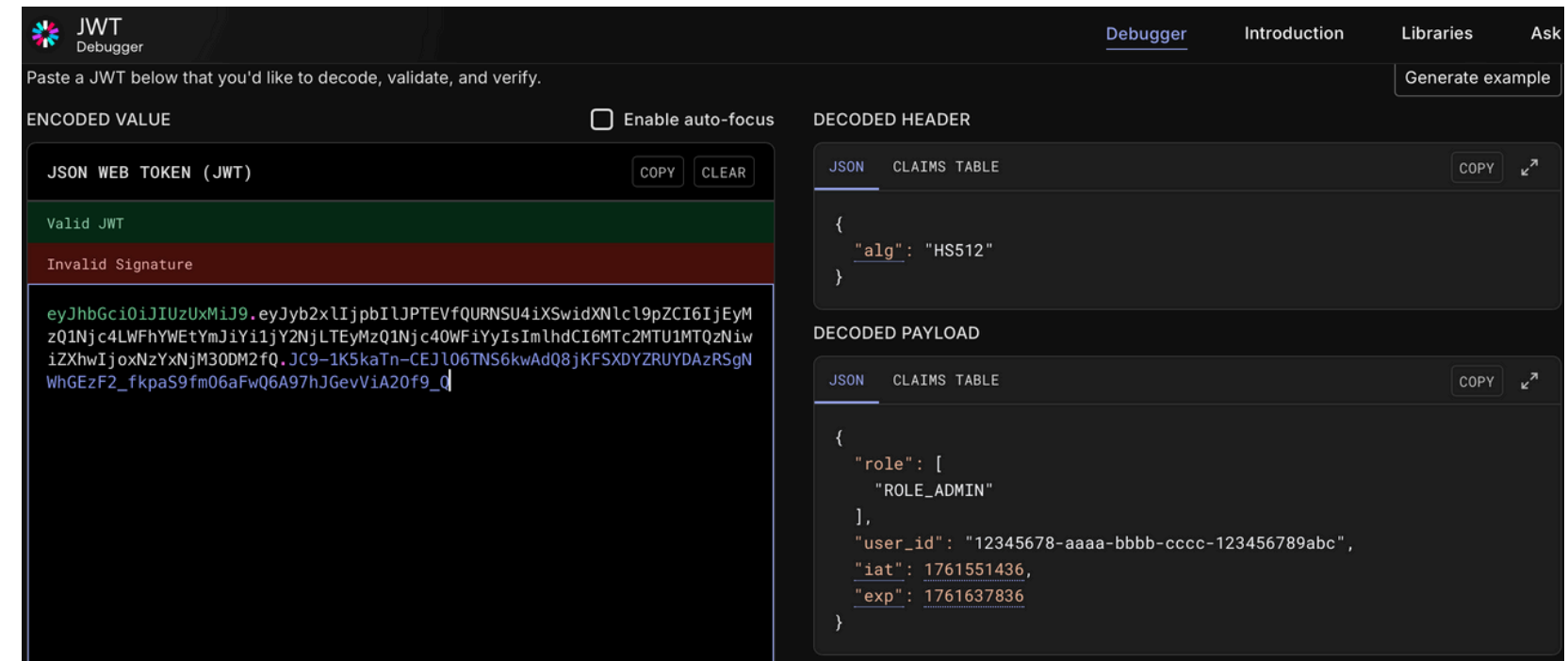
→ czBwdFVzZXI6czBwdDEyMzQ= (base64 인코딩/디코딩은 코드 한 줄이면 누구나 할 수 있어요.)

3. 그러면 요청 헤더는 이제 이렇게 구성됩니다.

→ Authorization: Basic czBwdFVzZXI6czBwdDEyMzQ=

Basic Authentication 그냥 인증 방법 중 하나

1. Authorization 헤더에서 Basic 키워드를 인식하고
2. 뒤에 있는 값을 base64 디코딩해서 → username:password 형태로 복원.
3. 이걸로 실제 DB의 유저 정보와 비교해 인증 여부를 판단

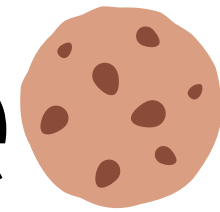


해커



1. Base64는 인코딩 TT 암호화가 아니라서 누구나 디코딩가능
2. 매 요청마다 ID, PW 전송 (http통신은 stateless)
3. 중간에 탈취되면 도용

Cookie



무엇이 문제냐,,



1. ~~Base64는 인코딩~~ ~~암호화가 아니라서~~ ~~누구나 디코딩가능~~
2. 매 요청마다 ID, PW 전송 (http통신은 stateless)
3. 중간에 탈취되면 도용

나: 아니 저번에 내가 말해줬잖아
기억 안 나? 내 말에 대답까지 했었잖아

👤 클라이언트: 안녕하세요, 로그인할게요. 제 이름은 지영입니다.

💻 서버: 반갑습니다. 인증되었습니다.

👤 클라이언트: 좋아요, 내 정보 보여주세요.

💻 서버: 누구시죠?

→ 서버가 클라이언트의 상태를 기억하지 않는다ㅠㅠ



Cookie🍪

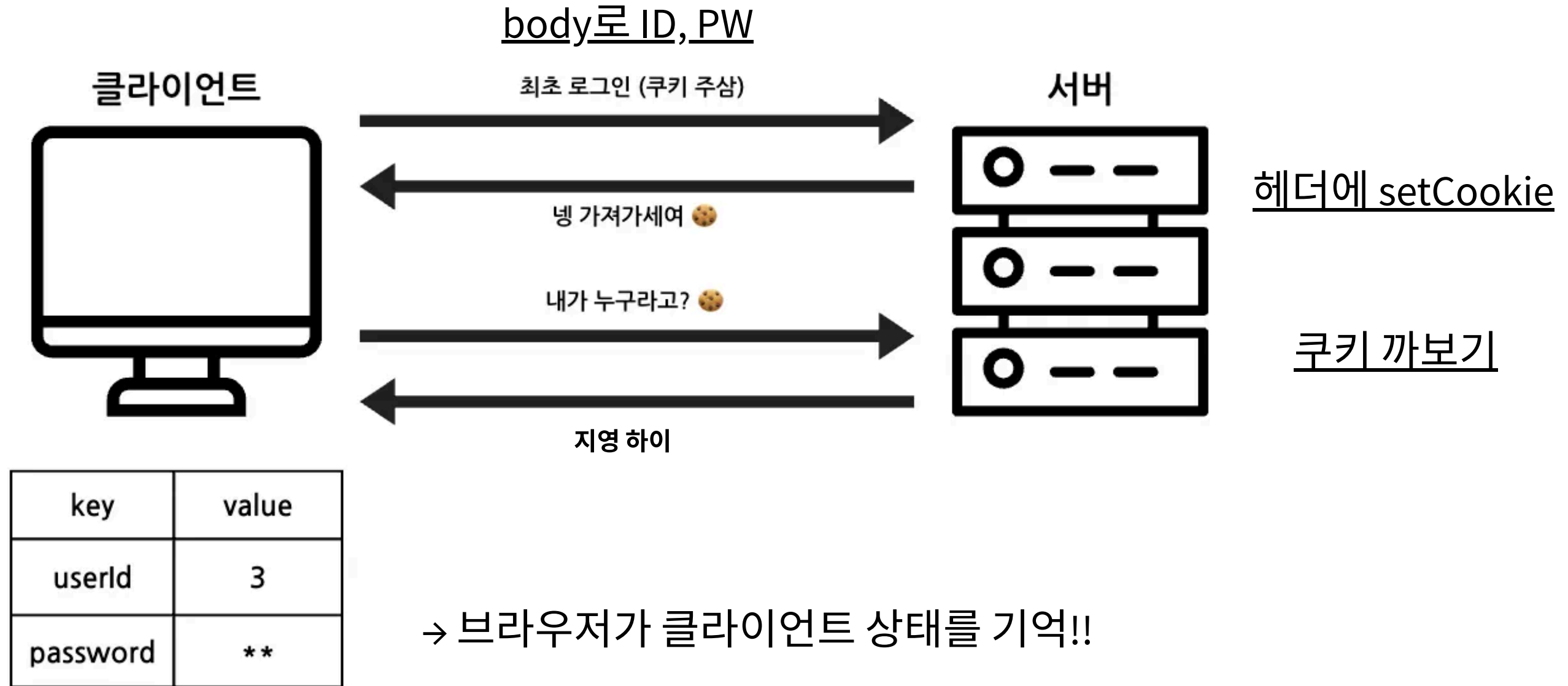
지금 이 요청을 보낸 사람이, 방금 로그인한 그 사람이 맞는지를 기억하기 위한 방법이 필요!!!

- 쿠키란 브라우저에 저장되어 있는 **key** 와 **value** 로 이루어진 작은 크기의 문자열
- 서버는 상태를 가지면 안되잖아. 그럼 차라리 **클라이언트에서 이를 보관**할까?
- 브라우저 **어딘가에** 인증 정보에 대해 저장해두고, 요청을 보낼 때마다 꺼내 쓴다면,
매 요청마다 아이디랑 비밀번호를 같이 주는 비효율적인 구조를 안 가져도 되겠네?

key	value
userId	3
password	***

쿠키flow🍪

브라우저가 자동으로 Cookie를
헤더에붙여서 요청(신경안써도됨)



1. 클라이언트는 로그인을 시도합니다.
2. 서버에서는 해당 정보를 확인하고, 인증되었다면 쿠키를 내려줘요.(key-value형태!)
3. 브라우저는 받은 쿠키를 저장합니다.
4. 이후의 요청부터는 자동으로 쿠키를 포함해서 요청합니다.
5. 서버에서는 쿠키에 담긴 정보를 확인하고 요청의 주체를 식별합니다.

쿠키의 핵심! 🍪

브라우저에 저장 (client측)

→ 왜?? 서버는 상태를 가지면 안되니까 브라우저에 인증 정보 저장했다가 요청마다 자동으로 서버로 보내줘서 확인하자

한 번 저장해두면, 이후 같은 도메인으로 요청을 보낼 때마다 **자동으로 그 쿠키를 헤더**에 실어줌

✓ 서버가 쿠키 설정하는 방식 (Set-Cookie)

```
Set-Cookie: key=value; Path=/; HttpOnly; Secure; Max-Age=3600
```

- `key=value` : 저장할 데이터
- `Path` : 어떤 경로에 대해 쿠키를 포함할지
- `HttpOnly` : JavaScript로 접근 못 하게 (XSS 보호)
- `Secure` : HTTPS에서만 전송
- `Max-Age` : 쿠키 유효 시간 (초 단위)

XSS(크로스 사이트 스크립팅)

공격자가 웹 페이지에 악성 자바스크립트를 삽입해서
사용자의 브라우저에서 실행되게 만들고
쿠키, 세션, 토큰 같은 민감한 정보를 탈취하는 공격 방식

쿠키가 탈취 당하지 않도록 예방하는 방법

HttpOnly

자바 스크립트에서 건드릴수 없다.(XSS)

Secure

Https에서만 사용가능 → http에선 못씀

SameSite

은행 사이트에 로그인한 상태에서 악성 사이트가 URL를 변경해서 악성 사이트로 넘어가는 요청을 보내면 브라우저는 자동으로 쿠키를 붙여서 요청을 보내버림

쿠키의 보안과 **CSRF(Cross-Site Request Forgery) 공격** 방지를 위해 도입된 속성
다른 사이트에서 온 요청에는 쿠키를 자동으로 붙이지 않게 할 수 있음.

Strict	완전 차단	오직 같은 도메인 요청일 때만 쿠키 전송
Lax (기본)	제한적 허용	링크 클릭 등 GET 네비게이션 요청만 허용
None	완전 허용	모든 요청에서 전송 가능 (단, Secure 필수)

Session

세션의 등장

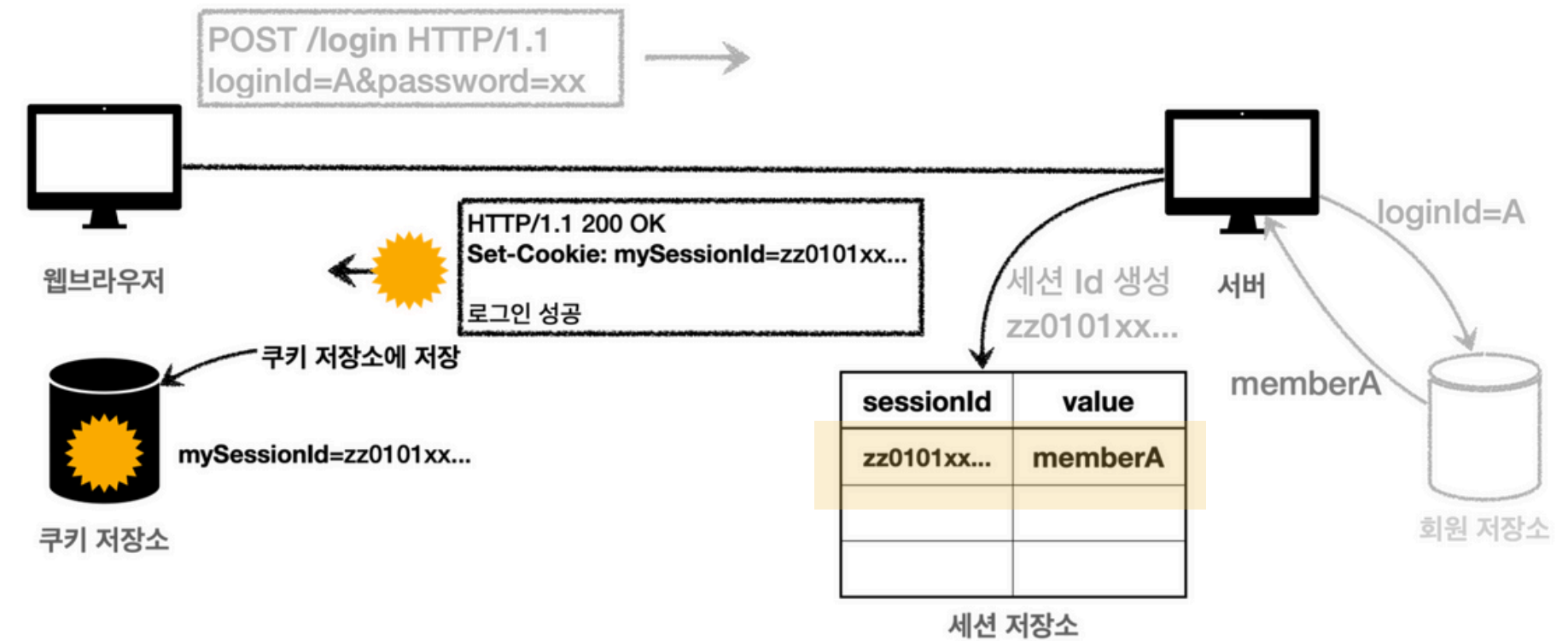
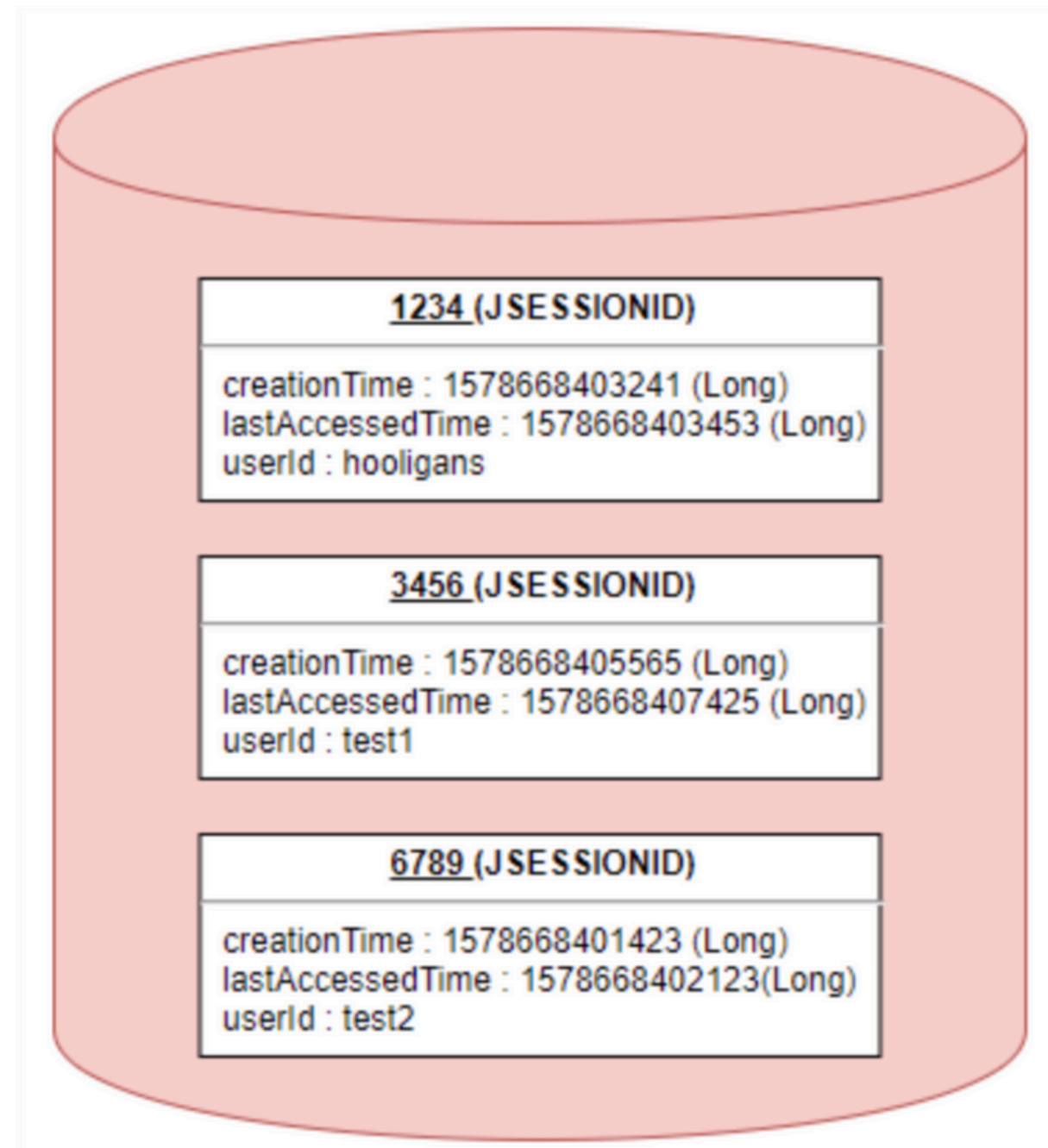


1. 매 요청마다 ID, PW 전송 (http통신은 stateless)
2. 중간에 탈취되면 도용

쿠키는 **브라우저**를 통해 관리되기 때문에 상대적으로 보안에 취약하고, 많은 보안 위협에 노출
어차피 인증 정보를 계속 보내야 한다면... 그걸 클라이언트가 아니라 **서버**가 기억하면 안 될까?

세션이란, **서버가 클라이언트의 정보**를 기억하기 위한 방법

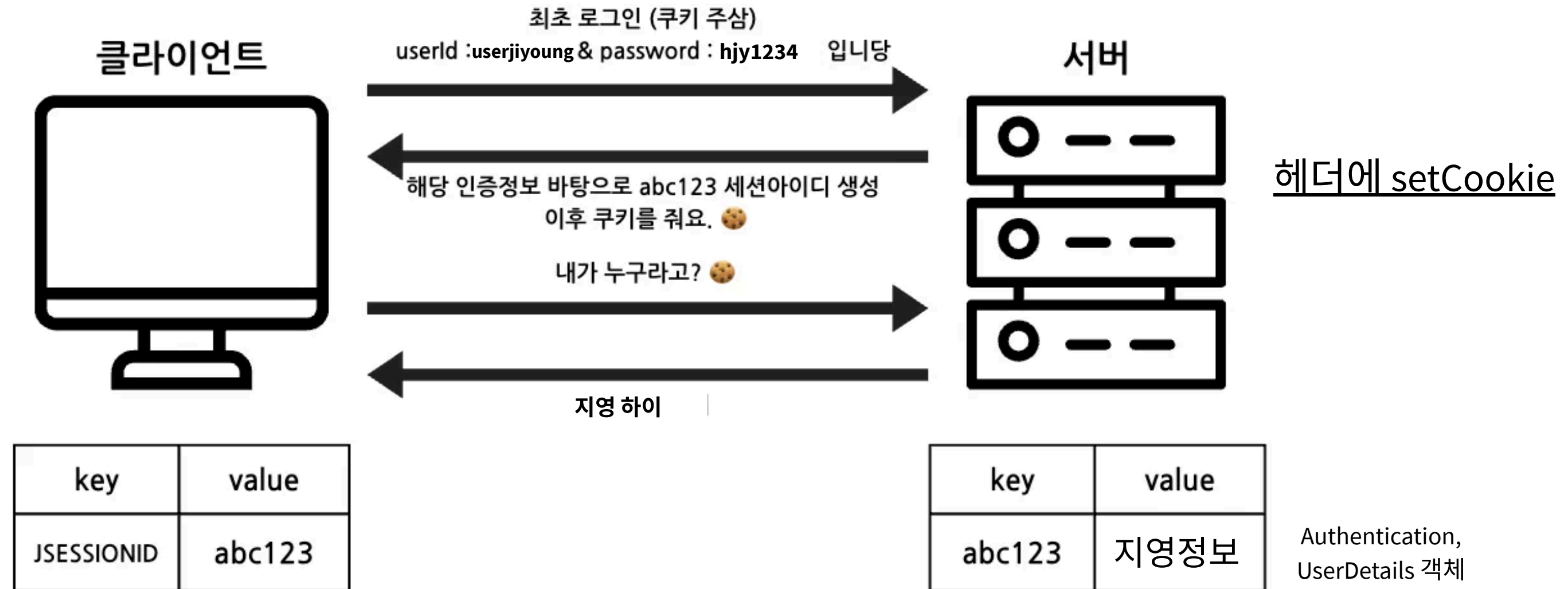
세션



세션 ID를 세션 저장소(서버)에 저장
해당 ID를 바탕으로 유저 정보를 저장
쿠키로 세션ID 전달

세션의 등장

1. 클라이언트가 로그인 성공 시, 서버는 임의의 문자열(세션 ID) 을 하나 생성해서
2. 클라이언트에게 쿠키 형태로 내려줍니다. (Set-Cookie: JSESSIONID=abc123)
3. 서버는 이 세션 ID를 기준으로, 메모리나 DB 등 어딘가에 유저 정보(상태) 를 저장해둬요.
4. 클라이언트는 이후 요청마다 이 세션 ID 쿠키만 보내면, 서버는 그걸 기반으로 유저 정보를 찾아서 인증을 처리합니다.



쿠키의 동작 방식에서 서버가 랜덤 sessionId를 만들어서 세션 저장소(= 서버)에 저장을 하고
클라이언트한테 넘겨주자 (key-value 쿠키)
-> 이 세션ID를 요청할 때 마다 보내서 유저 정보를 찾자

쿠키와 세션 차이!

구분	쿠키 Cookie	세션 Session
저장위치	클라이언트(로컬)	서버
라이프사이클 (만료시점)	쿠키 저장시 설정 (브라우저가 종료되더라도 만료시점이 지나지않으면 자동삭제되지 않음)	브라우저 종료시 삭제 (기간 별도 지정 가능)
보안	로컬에 저장되어 탈취,변조 위험 존재. 비교적 취약	서버에 저장되므로 안전
속도	빠름	제공받은 세션아이디를 이용해서 서버에서 다시 데이터를 참조해야 하므로 비교적 느림

세션의 장점! 그리고 또 문제 발생;;

✓ 장점

민감한 정보를 클라이언트에 저장하지 않아도 된다

서버가 유저 상태를 직접 관리할 수 있다

✓ 단점

서버에 저장소가 필요함 (메모리, DB 등)

서버가 유저 상태를 기억해야 하므로 **Stateless 아키텍처**와 충돌

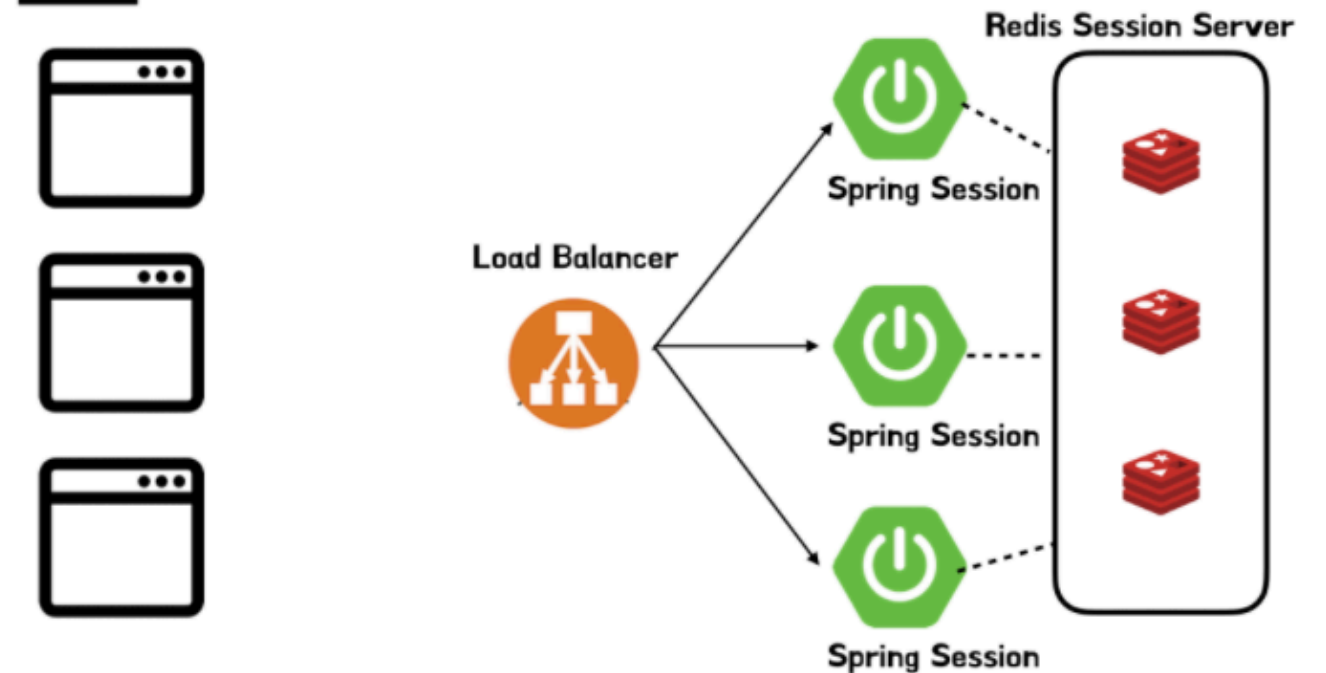
분산 서버 구조에서는 세션 공유 문제 발생 (→ Redis 같은 외부 저장소 필요)

→ 같은 유저임에도 불구하고 서버가 달라서 로그인 상태가 유지되지 않는 문제가 발생TT.

→ 외부 저장소에 병목현상TT

**세션 ID 자체가 탈취당하면, 그 세션 ID를 가진 사람은 마치 인증된 유저처럼 행동

Session Server 분리



Token

세션과 JWT의 비교

	세션	JWT
별개의 저장소가 필요한가	O	원칙적으로는 X
탈취의 위험성이 있는가	O	O
전달되는 형식	쿠키 : 세션ID	쿠키 : 리프레쉬 토큰 헤더 : 액세스 토큰 (상황마다 다름)
정보가 내부에 있는가	세션ID만 존재	토큰 내부에 존재

JWT(Json Web Token)는 이렇게 생겼어요

3개의 덩어리 → 이렇게 .(점) 으로 나뉜 3개의 파트를 Base64로 인코딩해서 이어 붙인 것

XXXX.YYYY.ZZZZ
Header.Payload.Signature



```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- alg 서명 알고리즘
- typ 토큰의 타입. 항상 JWT 라고 명시



```
{  
  "sub": "user",           // 주제(subject)  
  "userId": "202134650"   // 사용자 식별자  
  "role": "USER",        // 권한  
  "iat": 1716551471,      // 발급 시간 (issued at)  
  "exp": 1716555071      // 만료 시간 (expiration)  
}
```

Payload 부분은 누구나 디코딩합니다!
그러니까 절대 여기에 민감한 정보를 넣으면 X
사용자 ID 같은 최소한의 정보만 넣고, 나머지는 서버에서 조회

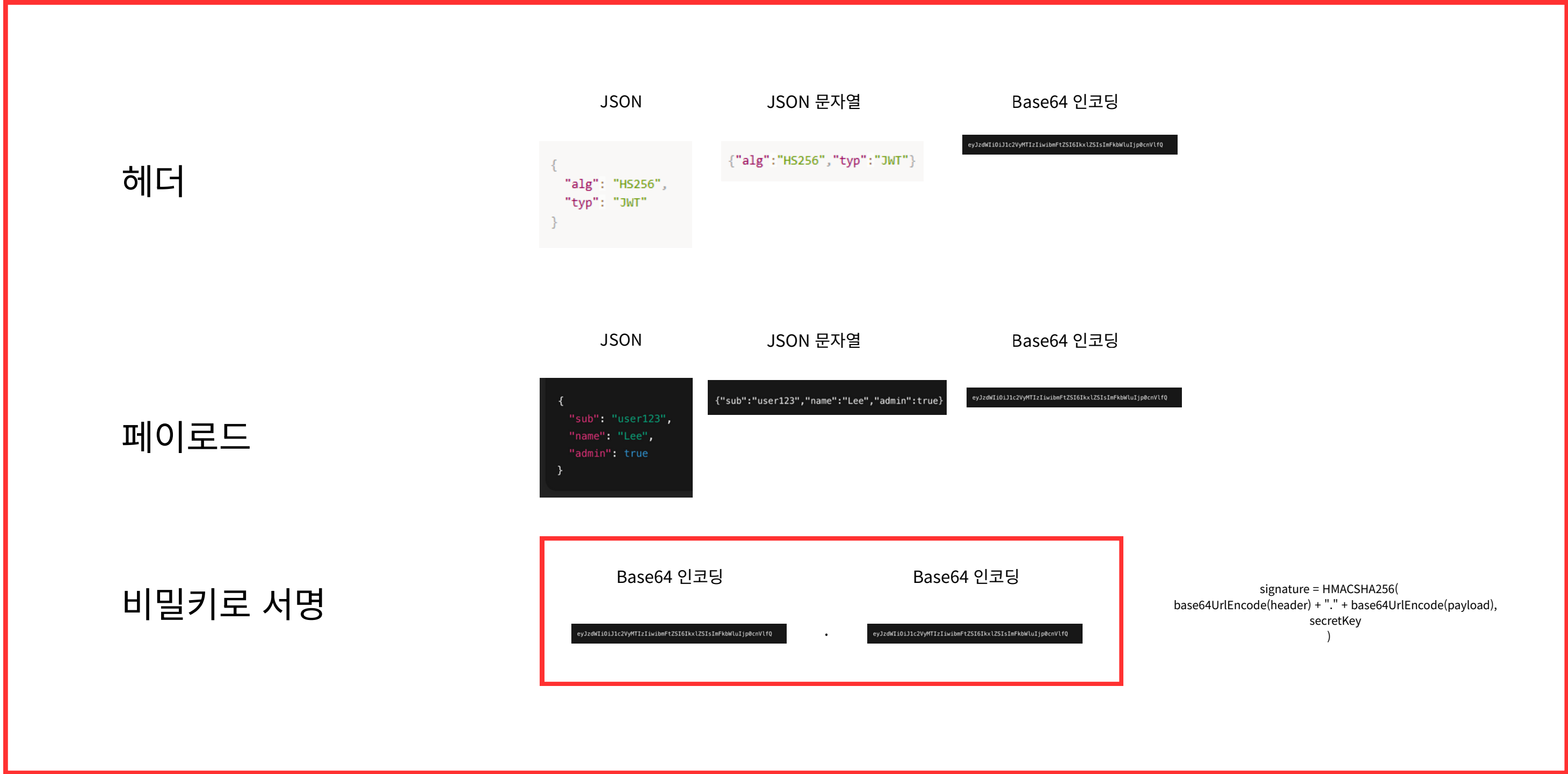
토큰은 내가(서버) 만든 게 맞아를 증명하기 위한 도장.
보통 application.yml 내에서 관리

```
HMACSHA256(  
  base64UrlEncode(header) + "." + base64UrlEncode(payload),  
  secret  
)
```

이 토큰이 정상적으로 만들어진 것인지 검증하는 역할
Header, Payload를 이어 붙인 후 비밀 키(secret) 를 이용해 서명하는 방식

Jwt는 이렇게 생겼어요

HEADER.PAYLOAD.SIGNATURE 모습으로 직렬화



JWT를 직접 발급 받아볼까요

```
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import java.util.Date;
import java.security.Key;

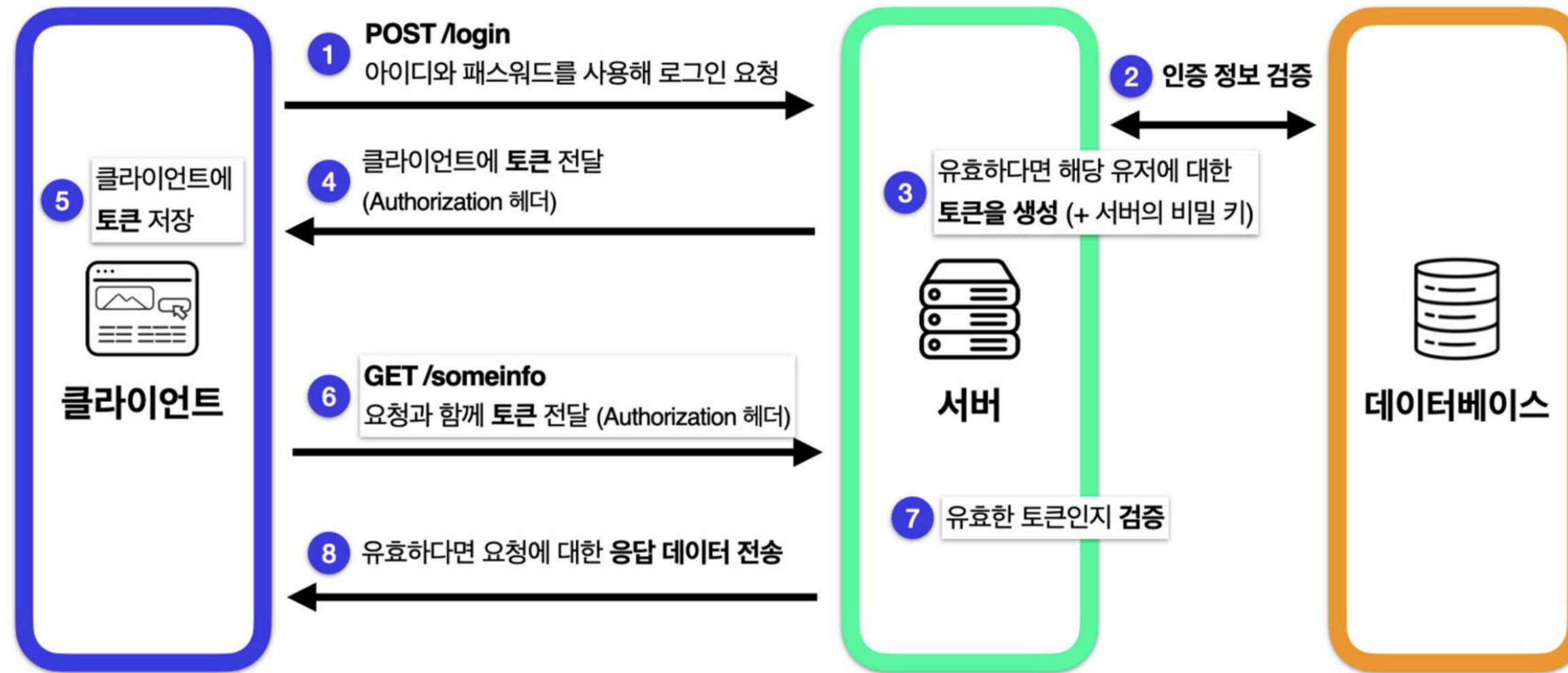
public class JwtUtil {
    private static final Key key =
Keys.hmacShaKeyFor("LeetsBackendKx5!93Jv#1Rz@lQwT9pXe3bD7sUaFzYt".getBytes());

    public static String generateToken(String userId) {
        long now = System.currentTimeMillis();

        return Jwts.builder()
            .setSubject(userId) // 토큰 주체
            .setIssuedAt(new Date(now)) // 발급 시간
            .setExpiration(new Date(now + 1000 * 60 * 60)) // 만료 시간 (1시간)
            .signWith(key, SignatureAlgorithm.HS256) // 서명
            .compact(); // JWT 문자열 생성
    }
}
```

Token 기반 인증방식 flow

→ 서버는 JWT에 담긴 정보만으로도 인증 여부를 판단할 수 있어요.
→ 별도 저장소도, 세션 상태도 필요 없습니다.



클라이언트가 로그인 요청

데베에 조회해서 어~ 있네~ 확인되면 **유저정보로 JWT(토큰)를 발급** - 어떤 정보로 만들지는 내 마음!

JWT를 그 자체로 인증 정보를 담은 채 클라이언트에게 전달 (헤더)

이후 요청마다 JWT를 헤더에 실어 보내기만 하면 끝

Access Token vs Refresh Token

Access Token vs Refresh Token 차이

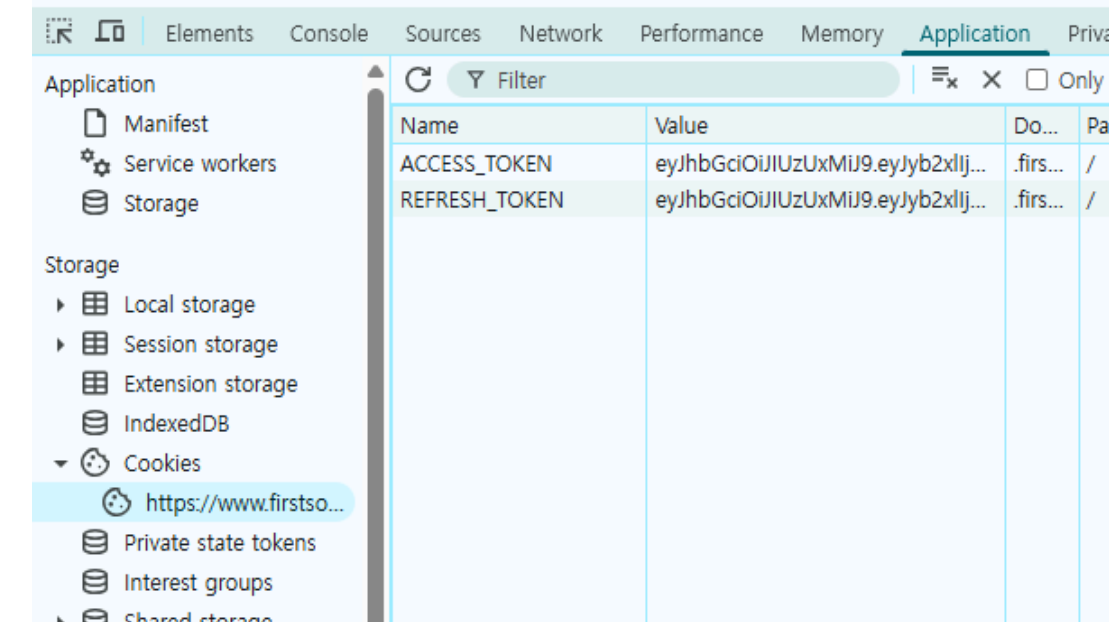
- 우리가 방금 token flow에서 봤던 그 토큰이 access Token입니다..!
- access Token 유효 시간이 짧습니다. (10분, 30분 등)

→ 만약 토큰이 탈취 당하더라도, 짧은 시간 안에 만료되게 만들기 위해서예요.

→ 우리가 아까 가졌던 여러가지 고민 중 하나죠? 인증 정보가 탈취되면 어떡하지..
그에 대한 약간의 해결책

Access Token vs Refresh Token 차이

그렇다면 Access Token의 유효 기간이 만료되면 어떻게 해야 할까요?
매번 다시 로그인하게 만들 수는 없겠죠. → 그래서 등장하는 것이 Refresh Token



Refresh Token은 Access Token이 만료됐을 때, **새로운 Access Token을 재발급 받기 위한 수단**

- 마찬가지로 로그인 시 클라이언트에게 함께 발급됨
 - Access Token과 달리, 일반적으로 이 토큰은 클라이언트가 API 요청에 직접 사용하지 않음
 - 대신 Access Token이 만료됐을 때에만 Refresh Token을 가지고 새로운 Access Token을 발급받기 위해 사용
- refresh Token으로 accessToken 재발급하는 API가 있어야하고 프론트에서 이를 호출해줘야 갱신됨
- 유효 기간이 깁니다. (2주, 1달 등)

```
String accessToken = jwtUtil.generateToken(user.getId());  
String refreshToken = jwtUtil.generateRefreshToken(user.getId());
```


JWT 검증 흐름

1. 클라이언트가 JWT 포함 요청을 보냄

```
GET /mypage HTTP/1.1
Authorization: Bearer <JWT>
```

어디에 저장하고 있다가???

- 쿠키/ 웹스토리지

2. 서버는 토큰을 구조적으로 분리

```
<Header>.<Payload>.<Signature>
```

3. signature 검증

```
Signature = HMACSHA256(
    base64UrlEncode(Header) + "." + base64UrlEncode(Payload),
    SECRET_KEY
)
```

4. 추가적인 유효성 검증

exp (Expiration Time): 만료된 토큰이면 거절

iat (Issued At): 발급 시간이 현재 시점보다 미래이면 이상한 토큰

nbf (Not Before): 아직 유효하지 않은 토큰

5. 검증 통과 → 인증 완료

6. 검증 실패 → 401 Unauthorized

이 유저는 인증 됐다!

**->토큰 내부에 있는 userId,role
같은 정보를 꺼내서 활용**

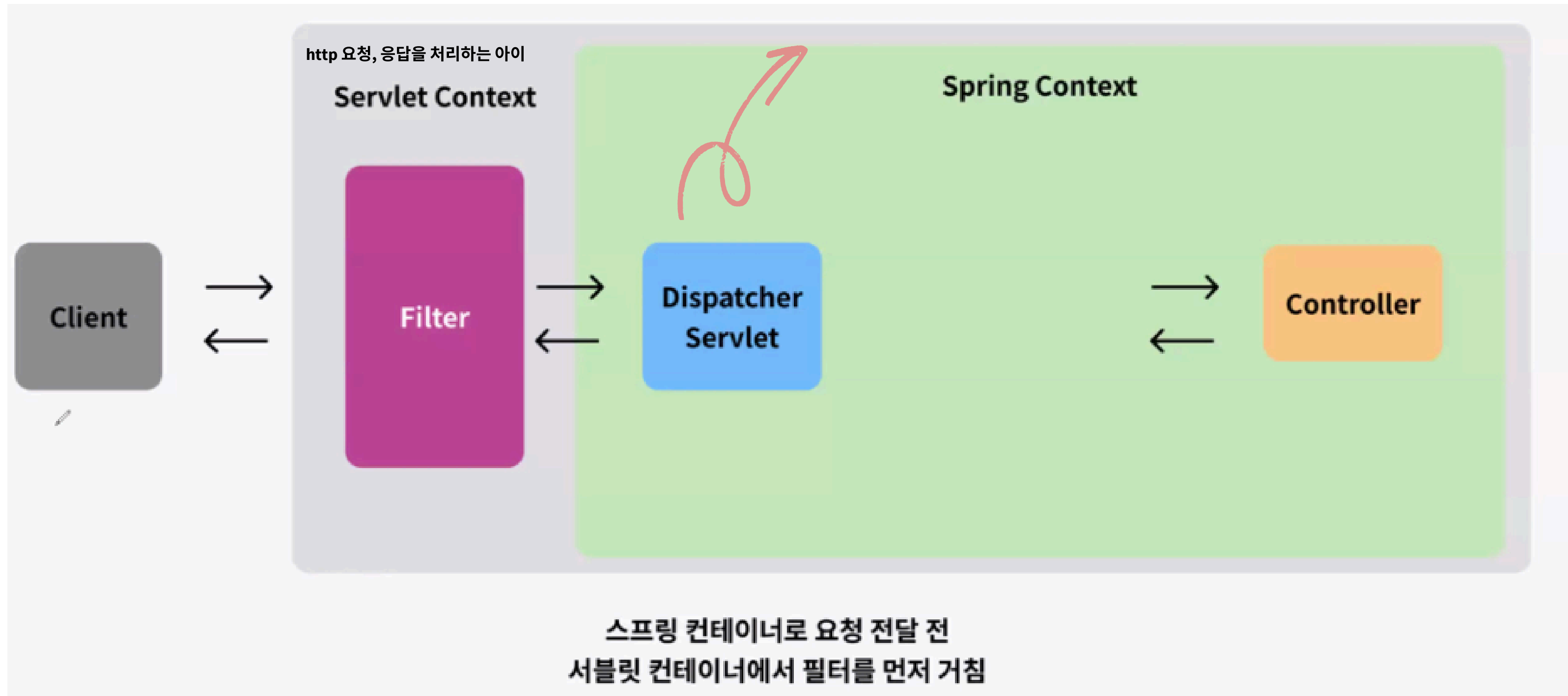
Spring Security

Spring Security

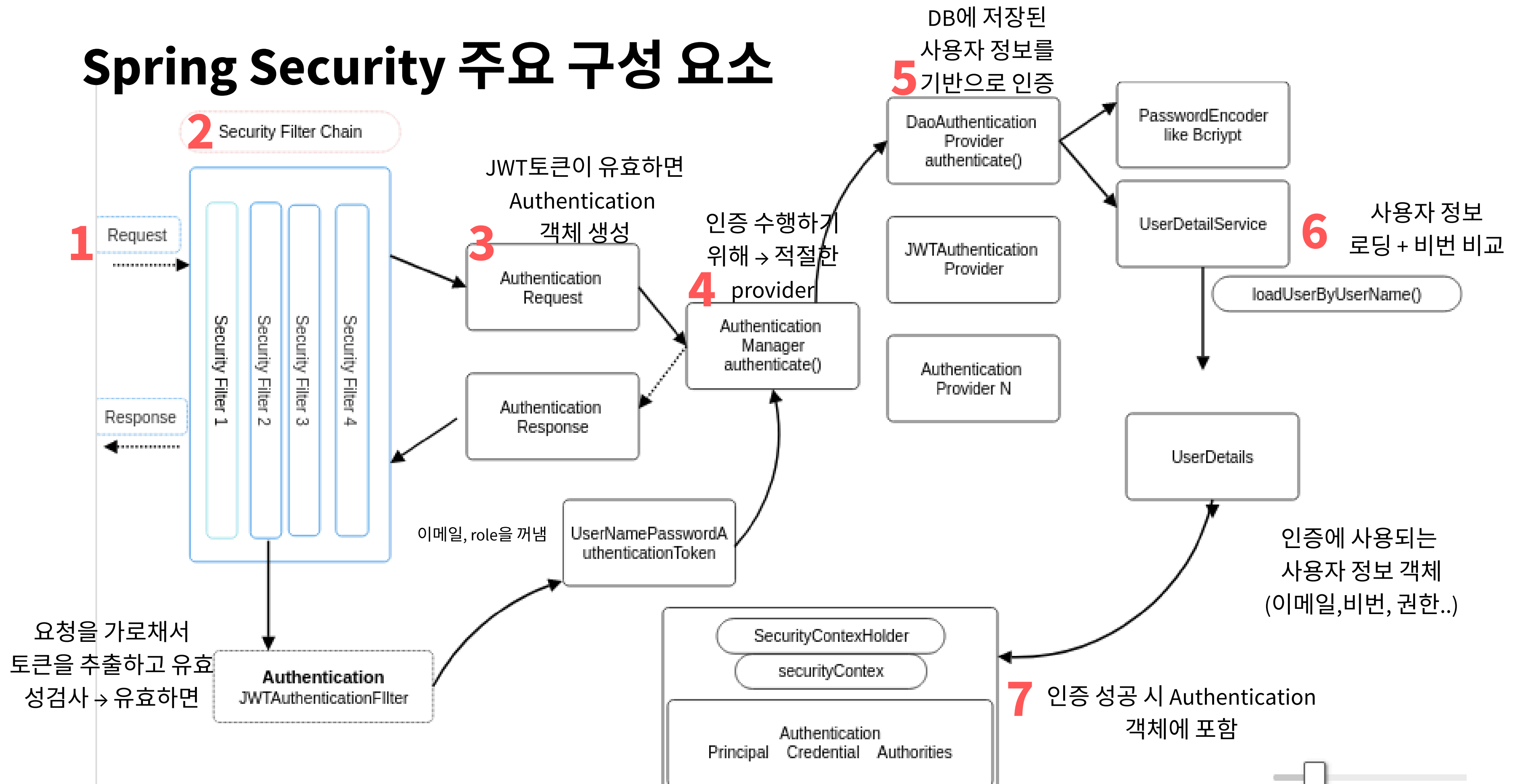
- Spring Security는 앞선 인증, 인가 + 보안 전반을 관리하는 스프링 프레임워크
- 세션 관리, 암호화, CORS 정책 적용,, 등등을 **필터**에 넣어줘서 꼼꼼하게 확인하자
- 쿠키·세션·JWT 같은 인증·인가를 직접 구현하면 보안 취약점과 유지보수 책임이 모두 개발자에게 돌아가지만, Spring Security는 검증된 **표준 방식**과 지속적인 업데이트로 안전하고 일관되게 처리해주기 때문에

요청 처리 흐름

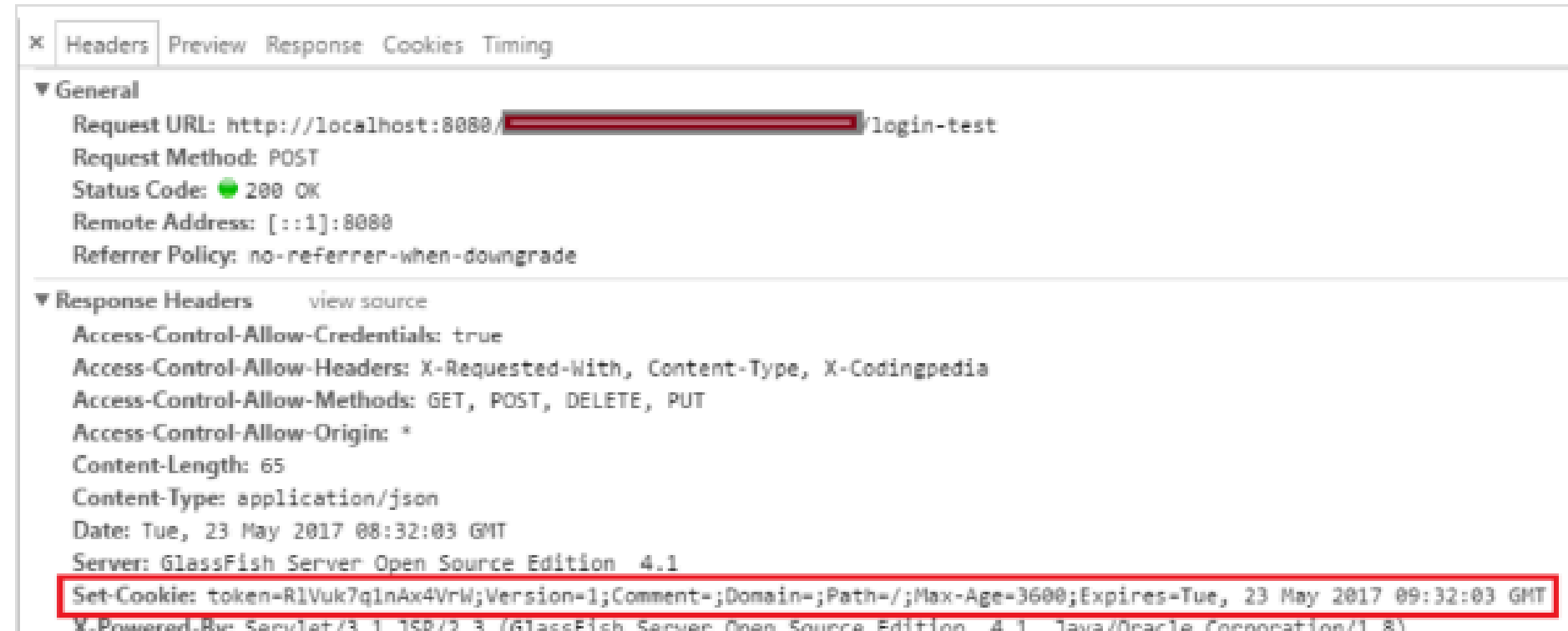
모든 들어오는 HTTP 요청을 중앙에서 받아서 처리하고,
적절한 Controller에게 라우팅하는 역할



Spring Security 주요 구성 요소



자세한 내용은...



× Headers Preview Response Cookies Timing

▼ General

Request URL: http://localhost:8080/[REDACTED]/login-test
Request Method: POST
Status Code: 200 OK
Remote Address: [::1]:8080
Referrer Policy: no-referrer-when-downgrade

▼ Response Headers view source

Access-Control-Allow-Credentials: true
Access-Control-Allow-Headers: X-Requested-With, Content-Type, X-Codingpedia
Access-Control-Allow-Methods: GET, POST, DELETE, PUT
Access-Control-Allow-Origin: *
Content-Length: 65
Content-Type: application/json
Date: Tue, 23 May 2017 08:32:03 GMT
Server: GlassFish Server Open Source Edition 4.1
Set-Cookie: token=RlVuk7q1nAx4VrW;Version=1;Comment=;Domain=;Path=/;Max-Age=3600;Expires=Tue, 23 May 2017 09:32:03 GMT
X-Powered-By: Servlet/3.1 JSP/2.3 (GlassFish Server Open Source Edition 4.1 Java/Oracle Corporation/1.8)

[Springboot] 1. Spring Security

Spring Security는 인증과 인가를 비롯한 보안 전반을 표준화해서 애플리케이션을 안전하게 보호하는 스프링 기반 프레임워크예요. 인증(Authentication)은 "누가 들어왔는가"를 확인하는 절차이고, 인가(Authorization)는 "들어온 사용자가 무

 velog.io

과제

회원가입, 로그인 구현하기

- spring security로 인증/인가 로직 구현
 - token 방식으로 구현 (accessToken, refreshToken)
 - 쿠키, 세션 방법은 상관없음 ->issue에 잘 정리해주기
 - 2주차에 걸쳐서 개발
-
- 이메일 로그인 + spring security로 인증/인가 로직 구현 → **11/4 (23:59)**
 - 카카오 로그인(oAuth) → **11/11 (23:59)**
 - 브라우저로 테스트 권장 → 화면 캡처하기

과제

회원가입(/auth)

- 사용자는 이메일 주소 / 카카오 OAuth를 통해 회원가입을 진행할 수 있어야 합니다.
- 사용자는 비밀번호를 생성하여 회원가입을 진행할 수 있어야 합니다.
- 사용자가 입력한 이메일 주소와 닉네임은 시스템에 이미 등록되어 있지 않아야 합니다.

로그인(/login)

- 사용자는 등록한 이메일 주소/ 카카오 로그인을 이용하여 로그인할 수 있어야 합니다.
- refresh token을 통해 새로운 access token을 발급받을 수 있어야 합니다.